FlipClip test: Part 2 and Part 3 answers, Robin Hmaidan

Robin Hmaidan · 1 Nov 2026

- **GitHub:** github.com/Robin-Hmaidan/flipclip
- **Live demo:** marvelous-pronto-unwieldy.ngrok-free.dev. It runs on my own PC, so the first visit shows an ngrok notice: click **Visit Site**.

Part 2: Time to Ship Safely

Caption Generator pre-deployment review. An internal tool: a logged-in user uploads a video, types a short description, and gets a social media caption from OpenAI (`gpt-4o-mini`). One Express 4 server (`backend/server.js`), MySQL (`config/schema.sql`), one vanilla-JS page (`frontend/index.html`), videos on local disk.

Summary

- **Verdict: not ready to go live.** There are several proven, unauthenticated paths to full data theft and a stolen API key. I'd block the launch on findings 1 to 5: each is easy to exploit, needs no special access, and leads to full data loss, remote file theft or a stolen API key. Findings 6 to 8 also need fixing before launch, and finding 9 means the app would not work in production as it stands.
- **Day one, fix 1:** real authentication on every endpoint, with the user ID taken from the session (finding 1). Today anyone can read or delete anyone's data without a password.
- **Day one, fix 2:** parameterise every query and serve files by ID with an ownership check (findings 2 and 3). Both are confirmed: login bypass and arbitrary file read.
- **Day one, fix 3:** rotate the OpenAI key, DB password and admin password, and move them to env vars (finding 4). They are committed to git and the running app hands them out.
- **After launch:** error and spend monitoring with OpenAI budget alerts, automated DB backups with a tested restore, and scheduled key rotation.

How I checked. I read all five files, resolved the dependencies into a lockfile (no install, no scripts) and ran `npm audit`. To confirm the three findings I was unsure of, I ran the app in two throwaway Docker containers (MySQL and Node, bound to localhost, using the fake OpenAI key already in the code, no real API calls), then deleted them. Findings marked **Confirmed** show the exact request I sent; the rest come from reading the code.

1. No real authentication: anyone can read, change or delete anything

Critical Found by code reading · `backend/server.js:64, 88, 138, 151, 164, 173`; `frontend/index.html:133-142`

- **What's wrong:** The login "token" is just `base64(username)`, and no endpoint checks it. Every endpoint trusts a `userId` or `videoId` sent by the client. `GET /api/all-users` has no protection at all. "Logged in" only exists in the browser's `localStorage`.
- **Why it matters:** Anyone who can reach the URL can list every user, read any user's videos and captions, delete any video, and call OpenAI on the company's account, with no password.
- **Fix:** Server-side sessions (httpOnly, Secure cookie) or properly signed JWTs, with middleware on every `/api` route. Take the user ID from the session, never the request, and check ownership before every read or delete. Remove `/api/all-users` or make it admin-only.

2. SQL injection in every query

Critical **Confirmed** (login bypass) · `backend/server.js:53, 75, 91, 121, 141, 154, 176, 192`

- **What's wrong:** Every query is built by pasting user input into the SQL string.
- **Why it matters:** A crafted username logs you in as admin without a password. The `:userId` and `:videoId` URL parameters allow `UNION` queries that can dump the whole database. Stacked queries are off by default in mysql2, so this is a data theft risk rather than a "drop table" risk. It also breaks normal use: at line 121 any caption containing an apostrophe ("it's", "don't") breaks the INSERT, so the user gets a 500 and the caption is lost after the OpenAI call has been paid for.

- **Confirmed with:**

  ```
  POST /api/login
  {"username":"admin' -- -","password":"wrongpassword"}
  ```

  Response: `HTTP 200` with `{"success":true,"user":{"id":1,"username":"admin","role":"admin",...}}`, i.e. logged in as admin with a wrong password. The comment sequence after `admin'` comments out the password check. The response also contained the stored password in plain text (finding 5).

- **Fix:** Parameterised queries everywhere (`db.execute('... WHERE id = ?', [id])`), and validate that IDs are integers.

3. Path traversal: unauthenticated arbitrary file read

Critical **Confirmed** · `backend/server.js:204-213`

- **What's wrong:** `GET /api/files/:filename` passes the URL parameter straight into `path.join`. Express URL-decodes route parameters, so an encoded `../` escapes the uploads folder.
- **Why it matters:** With no login, an attacker can read `server.js` and lift every secret in finding 4, then read config, keys or other users' uploads anywhere on the host.
- **Confirmed with:**

  ```
  GET /api/files/..%2fserver.js
  ```

  Response: `HTTP 200` with the full `server.js` source (6,603 bytes), including the hard-coded OpenAI key and admin password. `GET /api/files/..%2f..%2f..%2f..%2f..%2fetc%2fpasswd` returned `/etc/passwd`, so this reads any file the Node process can reach, not just source code.

- **Fix:** Serve files by database ID after an ownership check. Resolve the path and reject it unless it stays inside the uploads folder. Better: move files to object storage behind short-lived signed URLs.

4. Secrets are in the code, and the API returns them

Critical Found by code reading · `backend/server.js:16, 29-30, 131-132, 218`; `README.md:35-38`

- **What's wrong:** The OpenAI key, the MySQL **root** password and the admin password are hard-coded and committed to git. When an OpenAI call fails, the error response sends the API key to the caller (line 132), and the log line above it prints the axios error object, `Authorization: Bearer` header included. The admin password is printed at startup (line 218). The default logins are in the README.
- **Why it matters:** An attacker can make OpenAI calls fail on purpose (e.g. flood the open endpoint until it hits rate limits) and collect the key from the response, or just read it via finding 3. The result is an uncapped bill and a key that must be replaced. Anyone with repo access also gets DB root. The DB root password and the `admin` password are the same (`admin123`).
- **Fix:** Rotate all three secrets now; the old values stay in git history. Load secrets from env vars or a secrets manager. Return generic errors and log only `error.message`. Connect as a least-privilege DB user, not root. Remove the seed accounts from production.

5. Passwords stored in plain text and sent back to the browser

Critical Found by code reading (plain-text password seen in the finding 2 response) · `config/schema.sql:7, 28-29`; `backend/server.js:61`; `frontend/index.html:63`

- **What's wrong:** Passwords are stored in plain text. Login returns the whole user row, password included (`SELECT *`), and the frontend saves it in `localStorage`.
- **Why it matters:** With finding 1 or 2, one request leaks every user's password, and staff reuse passwords elsewhere. Any XSS (finding 6) can read the password straight from `localStorage`.
- **Fix:** Hash with bcrypt or argon2. Select named columns and never return the password. Store a session on the client, not the user object.

6. Stored XSS: filenames and AI output inserted as raw HTML

High Found by code reading · `frontend/index.html:73, 93, 108, 118-121`

- **What's wrong:** Filenames, the server's error message and the generated caption go into the page via `innerHTML`.

- **Why it matters:** A file named `<img src=x onerror=...>.mp4` runs JavaScript for anyone viewing that video list, and finding 1 lets an attacker upload into *someone else's* list. Model output is untrusted too: a prompt can make it return HTML. With the password in `localStorage` (finding 5), this is account takeover.
- **Fix:** Use `textContent` or build DOM nodes. Add a Content-Security-Policy header (e.g. with `helmet`).

7. Unlimited uploads and unlimited AI spending

High Found by code reading · `backend/server.js:37-46, 101-110`

- **What's wrong:** No upload size limit or file-type check. Files are saved under the original filename, so two users uploading `video.mp4` overwrite each other. The caption endpoint is open, unrate-limited and has no prompt length cap.
- **Why it matters:** One script can fill the disk and take the server down, or run up the OpenAI bill around the clock. Users can silently lose each other's videos.
- **Fix:** Set multer `limits.fileSize` and accept only video MIME types, checked against the file content, not the extension. Store files under random names (UUID). Add per-user rate limits and a prompt length cap. Set a spending limit in the OpenAI dashboard.

8. Dependencies with known vulnerabilities (1 critical, 8 high)

High Found by `npm audit` · `package.json:10-16`

- **What's wrong:** `npm audit` on the pinned versions reports 12 issues, including a **critical RCE in mysql2 2.3.0** (GHSA-fpw7-j2hg-69v5), a crash in multer's parser (dicer) that one malformed upload triggers, and DoS or prototype-pollution bugs in express, body-parser, qs and axios. There is no lockfile.
- **Why it matters:** Even with the code fixed, the server could be crashed remotely or attacked through its libraries.
- **Fix:** Upgrade to current majors (express 4.21+ or 5, mysql2 3.x, multer 2.x, axios 1.x). Replace `body-parser` with Express's built-in parser. Commit a lockfile and run `npm audit` in CI.

9. It won't actually work once deployed

Medium **Confirmed** (uploads fail) · `frontend/index.html:46`; `backend/server.js:10, 13-26, 39, 206`

- **What's wrong:**
  - The frontend calls `http://localhost:3000/api`, so in production every request goes to the visitor's own machine.
  - Multer writes to `./uploads` relative to the start folder, but the files endpoint reads from `backend/uploads`. Multer doesn't create a folder given by a destination function, and the repo has none, so every upload fails with ENOENT. **Confirmed:** `POST /api/upload` with a small file returned `HTTP 500` with `Error: ENOENT: no such file or directory, open 'uploads/sample.mp4'`, and no folder was created.
  - One MySQL connection, no pool, no reconnect, and the app keeps running if the DB connection fails.
  - Hard-coded port, no health check, plain HTTP.
- **Why it matters:** "Working locally. Ready to deploy" isn't accurate: users would get a broken app on day one, and the first time MySQL drops the idle connection every request fails until a restart.
- **Fix:** Relative API URLs (`/api`). Create the uploads folder at startup, with one absolute path for read and write. `mysql.createPool` plus a `/health` endpoint that checks the DB. Read `PORT` from the environment. Run behind a TLS reverse proxy, under a process manager or container with restarts. Local-disk videos also vanish on redeploy: another reason for object storage.

10. Data handling gaps

Low Found by code reading · `config/schema.sql:4-25`; `backend/server.js:57, 173-183`

- **What's wrong:**
  - Deleting a video removes the DB row but leaves the file on disk and its captions in the table. No foreign keys.
  - `username` isn't `UNIQUE`, so duplicate accounts are possible.
  - The login error response returns the raw SQL query (line 57).
  - User prompts go to OpenAI, and there is no retention policy for videos and captions.

- **Why it matters:** Users who delete content expect it gone. Leftover files are a data-protection risk (GDPR applies to the EU team), and orphaned rows pile up.
- **Fix:** Foreign keys with `ON DELETE CASCADE`, and delete the file in the same handler. `UNIQUE(username)`. Never return queries or stack traces to the client. Document what goes to OpenAI and set a retention period.

Order I'd fix things in

1. **Today:** take it offline, rotate the OpenAI key, DB password and admin password, and move secrets into env vars.
2. **Before launch:** fix findings 1 to 7 as one coordinated change to a 200-line file (sessions, parameterised queries, safe file serving, hashed passwords, `textContent`), then upgrade the dependencies (finding 8).
3. **Launch setup:** make the deployment work (finding 9) and close the data handling gaps (finding 10).

Part 3: Time to Dream

Blue Foxes runs a huge publishing machine: hundreds of channels across 10 platforms, with around a hundred pieces of content produced every day. At that volume, small manual steps repeated many times a day are where the time goes.

What I'd automate first

1. **Cutting long videos into shorts.** Someone watches a full special or match to find the best 30-60 seconds. It's the most time-heavy step, and FlipClip already automates the cutting.
2. **Packaging per platform.** Titles, captions, hashtags and translations are rewritten for each platform, channel and language. An LLM drafts, a human approves.
3. **Performance reporting.** View stats from all 10 platforms in one place, so the team sees which hooks work without opening ten dashboards.

The tool I'd build: Moment Finder

- **What it does:** An editor drops in a long video. The tool transcribes it, an LLM scores each segment for hook strength, humor, emotion or key plays, and it returns the top 10 moments, already cut and reframed to 9:16 by FlipClip. The editor approves, adjusts or rejects each one.
- **Why this one:** It removes the most expensive step, watching hours of footage, and fits catalog work like giving comedians' specials a second life on the Laugh Society channel.
- **Data-driven:** Every approve/reject and every clip's views feed back in, so the scoring learns what works per channel.
- **Done safely:** Spending caps on AI calls, login and roles, private file storage (the lessons from my Part 2 review), and a human always approves before publishing.
- **How I'd measure it:** Editor minutes per short, % of AI picks approved, and views of AI-picked shorts vs manual ones.